

Paradigmi di Programmazione - A.A. 2023-24

Esame Scritto del 20/05/2024

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione usando la strategia **call-by-name** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda x.x)((\lambda x.\lambda y.xy)y)$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

Una possibile soluzione:

$$\begin{aligned} & (\lambda x.x)((\lambda x.\lambda y.xy)y) \\ \rightarrow & \underline{(\lambda x.\lambda y.xy)y} \\ \equiv_{\alpha} & \underline{(\lambda x.\lambda k.xk)y} \\ \rightarrow & \lambda k.yk \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f x y =  
  if x then  
    match y with  
    | (0,b) -> b  
    | (_,b) -> x  
  else x;;
```

SOLUZIONE:

Struttura del tipo:

```
X -> Y -> RIS
```

Uso per convenzione X, Y, Z come variabili di tipo per i parametri x, y, z , RIS come variabile di tipo del risultato, e $A, B, C, \dots$ come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

```
X = bool           (da guardia dell'if)
Y = int * B        (da pattern matching)
RIS = B            (da primo caso del p.m.)
RIS = X            (da secondo caso del p.m. e ramo else)
```

Ne consegue:

```
X = bool
Y = int * bool
RIS = bool
```

Tipo inferito:

```
bool -> (int * bool) -> bool
```

Esercizio 3 [Punti 7]

Assumendo il seguente tipo di dato che descrive alberi n-ari di interi:

```
type ntrees =
| Node of int * ntrees list
```

Definire in OCaml le funzioni `contains` e `remove` con i seguenti tipi

```
contains : int -> ntrees -> bool
addchild : int -> ntrees -> ntrees
```

tali che:

- `contains` prende un intero n e un albero t , e restituisce `true` se n è presente in t . Restituisce `false` altrimenti
- `addchild` prende due interi n e m e un albero t , e restituisce un albero ottenuto aggiungendo ad ogni nodo etichettato con n in t , un figlio foglia etichettato con m . Se nessun nodo è etichettato con n , il metodo `addchild` restituisce un albero identico a t .

Ad esempio, considerando il seguente albero:

```
let t = Node (1, [
  Node (2, []);
  Node (3, [
    Node (4, []);
    Node (5, [])
  ])
])
```

```

    ]);
  Node (6,[
    Node (3,[
    ])
  ])
])

```

le funzioni danno i seguenti risultati

```

contains 4 t;; (* restituisce true *)          contains 7 t;; (* restituisce false *)
addchild 3 7 t;;
(* restituisce      Node (1, [
                        Node (2,[
                        Node (3,[
                                Node (7,[
                                Node (4,[
                                Node (5,[
                                ])
                        Node (6,[
                                Node (3,[
                                        Node (7,[
                                        ])
                                ])
                        ])
                        *)

```

SOLUZIONE:

Una possibile soluzione:

```

let rec contains n t=
  match t with
  | Node (x,tlist) -> n=x || List.exists (contains n) tlist;;

let rec addchild n m t =
  match t with
  | Node (x,tlist) ->
    if x=n then Node (x, (Node (m,[]))::(List.map (addchild n m) tlist))
    else Node (x, List.map (addchild n m) tlist)

```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con un nuovo tipo di dato che descrive valori definiti come coppie di valori più semplici, con le seguenti operazioni:

- **Pair(e1,e2)** definisce la coppia formata dai valori delle espressioni **e1** ed **e2**;
- **Frist(e)** fornisce il primo elemento della coppia descritta dall'espressione **e**
- **Second(e)** fornisce il secondo elemento della coppia descritta dall'espressione **e**

Ad esempio (in una sintassi concreta simile a OCaml):

```

Pair(3+2,false)          (* definisce la coppia (5,false) *)
First(Pair(3+2,false))    (* corrisponde a 5 *)
Second(Pair(3+2,false))   (* corrisponde a false *)
Pair(fun x -> x+1, fun x -> x-1) (* definisce una coppia contenente due funzioni *)

```

Inoltre, si introduca un costrutto `ComposeApply` che, data una coppia contenente due funzioni (`f1,f2`) e un parametro attuale `x`, restituisce il risultato dell'applicazione di `f2` al risultato dell'applicazione di `f1` a `x`.

- `ComposeApply(ep,earg)` dove `ep` valuta nella coppia (`f1,f2`) ed `earg` valuta nel valore `x`, calcola `f2(f1(x))`;

Un esempio articolato che mostra anche l'uso di `ComposeApply` (sempre in sintassi concreta simile a OCaml):

```
let f1 = fun x -> x+1 in
let f2 = fun x -> x*2 in
let p = (f1,f2) in
ComposeApply (p, 10);; (* il risultato è 22 *)
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.

SOLUZIONE:

Una possibile soluzione:

```
type exp = ...
  | Pair of exp * exp
  | First of exp
  | Second of exp
  | ComposeApply of exp * exp

type evT = ...
  | PairVal of evT * evT

let rec eval e s = match e with
  ...
  | Pair(e1,e2) -> PairVal (eval e1 s, eval e2 s)
  | First e -> (match (eval e s) with
    | PairVal (v1,v2) -> v1
    | _ -> failwith "Error")
  | Second e -> (match (eval e s) with
    | PairVal (v1,v2) -> v2
    | _ -> failwith "Error")
  | ComposeApply (ep,earg) ->
    let pairclosure = eval ep s in
    (match pairclosure with
    | PairVal (Closure(arg1,body1,fDecEnv1), Closure(arg2,body2,fDecEnv2)) ->
      let aval = eval earg s in
      let aenv1 = bind fDecEnv1 arg1 aval in
      let ris1 = eval body1 aenv1 in
      let aenv2 = bind fDecEnv2 arg2 ris1 in
      eval body2 aenv2
    | _ -> failwith "Error")
  )
```